DAM 0485 · Programación · Unidad Didáctica 4

Boletín 4.2: Nivel Intermedio (Resuelto) — Soluciones Comentadas

Soluciones completas, comentadas y optimizadas de los 16 ejercicios de nivel intermedio de la Unidad 4 en Java 25 LTS.

A continuación se presentan las soluciones comentadas de los 16 ejercicios de nivel intermedio de la Unidad 4, haciendo énfasis en la inmutabilidad, el respeto a los contratos de `Object`, la modularidad y los patrones de diseño orientados a objetos.

Solución Ejercicio 1: Sobrescritura Canónica de `toString()`, `equals()` y `hashCode()` en `Libro`

```
import java.util.Objects;

public class Libro {
    private String titulo;
    private String autor;
    private int paginas;

    public Libro(String titulo, String autor, int paginas) {
        this.titulo = Objects.requireNonNull(titulo, "El título no puede ser nulo");
        this.autor = Objects.requireNonNull(autor, "El autor no puede ser nulo");
        this.paginas = Math.max(paginas, 0);
    }

    public String getTitulo() { return this.titulo; }
    public String getAutor() { return this.autor; }
    public int getPaginas() { return this.paginas; }

    @Override
    public String toString() {
        return this.titulo + ", de " + this.autor + " (" + this.paginas + " páginas)";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true; // 1. Misma referencia en el Heap
        if (!(o instanceof Libro otro)) return false; // 2. Null o clase incompatible
        // 3. Igualdad semántica por título y autor
        return Objects.equals(this.titulo, otro.titulo)
                && Objects.equals(this.autor, otro.autor);
    }

    @Override
    public int hashCode() {
        // Obligatorio usar exactamente los mismos campos que en equals()
        return Objects.hash(this.titulo, this.autor);
    }

    public static void main(String[] args) {
        Libro a = new Libro("Ficciones", "Borges", 200);
        Libro b = new Libro("Ficciones", "Borges", 350); // Mismo título y autor, distintas
páginas
        Libro c = new Libro("El Aleph", "Borges", 200);  // Distinto título

        System.out.println(a); // Imprime: Ficciones, de Borges (200 páginas)

        // Caso 1: Igual título y autor, distintas páginas -> true
        System.out.println("¿a.equals(b)? " + a.equals(b));
        // Caso 2: Distinto título -> false
        System.out.println("¿a.equals(c)? " + a.equals(c));
        // Caso 3: Reflexividad -> true
        System.out.println("¿a.equals(a)? " + a.equals(a));
    }
}
```

Solución Ejercicio 2: Composición (`Motor` y `Vehiculo`)

```
public class Motor {
    private final String combustible;
    private final int potenciaCv;
    private boolean encendido;

    public Motor(String combustible, int potenciaCv) {
        this.combustible = combustible;
        this.potenciaCv = Math.max(potenciaCv, 1);
        this.encendido = false;
    }

    public void encender() { this.encendido = true; }
    public void apagar() { this.encendido = false; }
    public boolean isEncendido() { return this.encendido; }
    public int getPotenciaCv() { return this.potenciaCv; }
    public String getCombustible() { return this.combustible; }
}
```

```
public class Vehiculo {
    private final String matricula;
    private final String marca;
    private final Motor motor; // Composición

    public Vehiculo(String matricula, String marca, Motor motor) {
        this.matricula = matricula;
        this.marca = marca;
        this.motor = java.util.Objects.requireNonNull(motor, "El motor es obligatorio");
    }

    public void arrancar() {
        if (!this.motor.isEncendido()) {
            this.motor.encender();
            System.out.println("Vehículo " + this.matricula + " ha arrancado su motor.");
        }
    }

    public void detener() {
        this.motor.apagar();
        System.out.println("Vehículo " + this.matricula + " ha apagado su motor.");
    }

    @Override
    public String toString() {
        return String.format("Vehiculo[%s %s | Motor: %d CV %s | Estado: %s]",
                this.marca, this.matricula, this.motor.getPotenciaCv(),
                this.motor.getCombustible(), this.motor.isEncendido() ? "ENCENDIDO" : "APAGA
DO");
    }
}
```

Solución Ejercicio 3: La Clase `Racional` / `Fraccion` Inmutable con Simplificación MCD

```
import java.util.Objects;

public final class Fraccion {
    private final int numerador;
    private final int denominador;

    // Constructor por defecto: racional 1/1
    public Fraccion() {
        this(1, 1);
    }

    public Fraccion(int numerador, int denominador) {
        if (denominador == 0) {
            throw new IllegalArgumentException("El denominador no puede ser cero.");
        }

        // Normalización de signos
        if (denominador < 0) {
            numerador = -numerador;
            denominador = -denominador;
        }

        // Simplificación por Máximo Común Divisor (Euclides recursivo)
        int divisor = mcd(Math.abs(numerador), denominador);
        this.numerador = numerador / divisor;
        this.denominador = denominador / divisor;
    }

    private static int mcd(int a, int b) {
        return b == 0 ? a : mcd(b, a % b);
    }

    public int getNumerador() { return this.numerador; }
    public int getDenominador() { return this.denominador; }

    public Fraccion sumar(Fraccion otra) {
        int num = (this.numerador * otra.denominador) + (otra.numerador * this.denominador);
        int den = this.denominador * otra.denominador;
        return new Fraccion(num, den);
    }

    public static Fraccion sumar(Fraccion r1, Fraccion r2) {
        return r1.sumar(r2);
    }

    public Fraccion restar(Fraccion otra) {
        int num = (this.numerador * otra.denominador) - (otra.numerador * this.denominador);
        int den = this.denominador * otra.denominador;
        return new Fraccion(num, den);
    }

    public Fraccion multiplicar(Fraccion otra) {
        return new Fraccion(this.numerador * otra.numerador, this.denominador * otra.denomin
ador);
    }

    public Fraccion dividir(Fraccion otra) {
        if (otra.numerador == 0) {
            throw new ArithmeticException("No se puede dividir entre una fracción con valor
cero.");
```

```
        }
        return new Fraccion(this.numerador * otra.denominador, this.denominador * otra.numer
ador);
    }

    @Override
    public String toString() {
        return this.numerador + "/" + this.denominador;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Fraccion f)) return false;
        return this.numerador == f.numerador && this.denominador == f.denominador;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.numerador, this.denominador);
    }
}
```

Solución Ejercicio 4: Clase Inmutable Clásica `Coordenada` y `record CoordenadaGPS`

```
// 1. Clase inmutable tradicional (final class + atributos private final + sin setters)
public final class Coordenada {
    private final double x;
    private final double y;

    public Coordenada(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public double getY() { return this.y; }

    public double distanciaA(Coordenada otra) {
        return Math.sqrt(Math.pow(otra.x - this.x, 2) + Math.pow(otra.y - this.y, 2));
    }

    @Override
    public String toString() {
        return "(" + this.x + ", " + this.y + ")";
    }
}
```

```
// 2. Enfoque moderno con record de Java
public record CoordenadaGPS(double latitud, double longitud) {

    public CoordenadaGPS {
        if (latitud < -90.0 || latitud > 90.0) {
            throw new IllegalArgumentException("Latitud fuera de rango [-90, 90]: " + latitu
d);
        }
        if (longitud < -180.0 || longitud > 180.0) {
            throw new IllegalArgumentException("Longitud fuera de rango [-180, 180]: " + lon
gitud);
        }
    }

    public double distanciaKm(CoordenadaGPS destino) {
        final double RADIO_TIERRA_KM = 6371.0;
        double dLat = Math.toRadians(destino.latitud() - this.latitud);
        double dLon = Math.toRadians(destino.longitud() - this.longitud);

        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
                   Math.cos(Math.toRadians(this.latitud)) * Math.cos(Math.toRadians(destino.
latitud())) *
                   Math.sin(dLon / 2) * Math.sin(dLon / 2);

        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return RADIO_TIERRA_KM * c;
    }
}
```

Solución Ejercicio 5: El Patrón Singleton (`ContadorGlobal` y `GestorConfiguracion`)

```
// 1. Singleton con Inicialización Perezosa (Lazy Initialization)
public class ContadorGlobal {
    private static ContadorGlobal instancia;
    private int valor;

    private ContadorGlobal() {
        this.valor = 0;
    }

    public static ContadorGlobal getInstancia() {
        if (instancia == null) {
            instancia = new ContadorGlobal();
        }
        return instancia;
    }

    public void incrementar() {
        this.valor++;
    }

    public int getValor() {
        return this.valor;
    }

    public static void main(String[] args) {
        ContadorGlobal c1 = ContadorGlobal.getInstancia();
        ContadorGlobal c2 = ContadorGlobal.getInstancia();

        c1.incrementar();
        c1.incrementar();
        c2.incrementar();

        System.out.println("Valor desde c1: " + c1.getValor()); // 3
        System.out.println("Valor desde c2: " + c2.getValor()); // 3
        System.out.println("¿Son el mismo objeto (c1 == c2)?: " + (c1 == c2)); // true
    }
}
```

```
// 2. Singleton con Inicialización Temprana (Eager Initialization)
import java.util.HashMap;
import java.util.Map;

public class GestorConfiguracion {
    private static final GestorConfiguracion INSTANCIA = new GestorConfiguracion();
    private final Map<String, String> propiedades;

    private GestorConfiguracion() {
        this.propiedades = new HashMap<>();
        this.propiedades.put("app.nombre", "SistemaDAM");
        this.propiedades.put("app.version", "2.5.0");
    }

    public static GestorConfiguracion getInstance() {
        return INSTANCIA;
    }

    public void setPropiedad(String clave, String valor) {
        this.propiedades.put(clave, valor);
    }

    public String getPropiedad(String clave, String valorPorDefecto) {
        return this.propiedades.getOrDefault(clave, valorPorDefecto);
    }
}
```

Solución Ejercicio 6: La Clase `Mochila` con Copias Defensivas (`.clone()` y `Arrays.copyOf()`)

```
import java.util.Arrays;

public class Mochila {
    private final String[] objetos;

    public Mochila(String[] objetos) {
        // Copia defensiva al entrar (evita que cambios en el array original alteren la moch
ila)
        this.objetos = (objetos != null) ? objetos.clone() : new String[0];
    }

    public String[] getObjetos() {
        // Copia defensiva al salir (evita que quien recibe el array modifique el interno)
        return this.objetos.clone();
    }

    public static void main(String[] args) {
        String[] equipo = {"Linterna", "Cantimplora", "Mapa"};
        Mochila m = new Mochila(equipo);

        // Intentamos sabotear la mochila desde fuera de dos formas distintas:
        equipo[0] = "ROTO";
        m.getObjetos()[1] = "VACÍO";

        // Comprobamos que el estado interno de la mochila sigue 100% intacto:
        System.out.println("Contenido seguro de la mochila: " + Arrays.toString(m.getObjetos
()));
        // Imprime: [Linterna, Cantimplora, Mapa]
    }
}
```

Solución Ejercicio 7: Enumeración Rica `NivelSeveridad`

```
public enum NivelSeveridad {
    DEBUG(1, "DEB", " [36m"),
    INFO(2, "INF", " [32m"),
    WARN(3, "WRN", " [33m"),
    ERROR(4, "ERR", " [31m"),
    FATAL(5, "FTL", " [35m");

    private final int nivelNumerico;
    private final String prefijo;
    private final String codigoColorAnsi;

    NivelSeveridad(int nivelNumerico, String prefijo, String codigoColorAnsi) {
        this.nivelNumerico = nivelNumerico;
        this.prefijo = prefijo;
        this.codigoColorAnsi = codigoColorAnsi;
    }

    public int getNivelNumerico() { return this.nivelNumerico; }
    public String getPrefijo() { return this.prefijo; }
    public String getCodigoColorAnsi() { return this.codigoColorAnsi; }

    public boolean esCritico() {
        return this.nivelNumerico >= 4;
    }

    public boolean esMayorOIgualQue(NivelSeveridad otro) {
        return this.nivelNumerico >= otro.nivelNumerico;
    }
}
```

Solución Ejercicio 8: `CuentaBancaria` con Validación de Excepciones e Historial

```
import java.time.LocalDateTime;
import java.util.Arrays;

public class CuentaBancaria {
    public record Transaccion(LocalDateTime fechaHora, String tipo, double importe, double s
aldoPosterior) {}

    private final String iban;
    private final String titular;
    private double saldo;
    private final Transaccion[] historial;
    private int contadorTransacciones;

    public CuentaBancaria(String iban, String titular, double saldoInicial) {
        if (titular == null || titular.trim().isEmpty()) {
            throw new IllegalArgumentException("El titular es obligatorio.");
        }
        if (saldoInicial < 0) {
            throw new IllegalArgumentException("El saldo inicial no puede ser negativo.");
        }
        this.iban = iban;
        this.titular = titular.trim();
        this.saldo = saldoInicial;
        this.historial = new Transaccion[10];
        this.contadorTransacciones = 0;
    }

    public void ingresar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a ingresar debe ser mayor que
0.");
        }
        this.saldo += cantidad;
        registrar("INGRESO", cantidad);
    }

    public void retirar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a retirar debe ser mayor que
0.");
        }
        if (cantidad > this.saldo) {
            throw new IllegalArgumentException("Saldo insuficiente. Saldo actual: " + this.s
aldo + "€");
        }
        this.saldo -= cantidad;
        registrar("RETIRADA", cantidad);
    }

    private void registrar(String tipo, double importe) {
        Transaccion t = new Transaccion(LocalDateTime.now(), tipo, importe, this.saldo);
        if (this.contadorTransacciones < 10) {
            this.historial[this.contadorTransacciones++] = t;
        } else {
            System.arraycopy(this.historial, 1, this.historial, 0, 9);
            this.historial[9] = t;
        }
    }

    public Transaccion[] getUltimosMovimientos() {
```

```
        return Arrays.copyOf(this.historial, this.contadorTransacciones);
    }

    public double getSaldo() { return this.saldo; }
    public String getTitular() { return this.titular; }

    public static void main(String[] args) {
        CuentaBancaria cuenta = new CuentaBancaria("ES01-2000", "Ana", 100.0);
        try {
            cuenta.ingresar(50.0);
            System.out.println("Saldo tras ingreso: " + cuenta.getSaldo() + "€");
            cuenta.retirar(200.0); // Lanzará IllegalArgumentException
        } catch (IllegalArgumentException e) {
            System.out.println("Operación rechazada: " + e.getMessage());
        }
    }
}
```

Solución Ejercicio 9: Validación Compacta con `record Usuario`

```
public record Usuario(String username, String email, int edad) {

    public Usuario {
        if (username == null || !username.matches("^[a-zA-Z0-9_]{4,15}$")) {
            throw new IllegalArgumentException("El username debe tener entre 4 y 15 caracter
es alfanuméricos.");
        }
        if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,6}$")) {
            throw new IllegalArgumentException("El formato del correo electrónico no es váli
do.");
        }
        if (edad < 18) {
            throw new IllegalArgumentException("El usuario debe ser mayor de edad (≥ 18 año
s): " + edad);
        }
    }
}
```

Solución Ejercicio 10: Enumeración de Divisas con Conversión

```
public enum Divisa {
    EUR(1.0, "€"),
    USD(1.08, "$"),
    GBP(0.85, "£"),
    JPY(162.50, "¥");

    private final double tasaRespectoEuro;
    private final String simbolo;

    Divisa(double tasaRespectoEuro, String simbolo) {
        this.tasaRespectoEuro = tasaRespectoEuro;
        this.simbolo = simbolo;
    }

    public double getTasaRespectoEuro() { return this.tasaRespectoEuro; }
    public String getSimbolo() { return this.simbolo; }

    public double convertirA(double cantidad, Divisa divisaDestino) {
        if (cantidad < 0) throw new IllegalArgumentException("La cantidad no puede ser negat
iva.");
        // Paso 1: Convertir a Euros base
        double euros = cantidad / this.tasaRespectoEuro;
        // Paso 2: Convertir de Euros a divisa destino
        return euros * divisaDestino.tasaRespectoEuro;
    }
}
```

Solución Ejercicio 11: Encapsulación de Matriz con `Matriz2D`

```
import java.util.Arrays;

public final class Matriz2D {
    private final double[][] datos;
    private final int filas;
    private final int columnas;

    public Matriz2D(int filas, int columnas) {
        if (filas <= 0 || columnas <= 0) throw new IllegalArgumentException("Dimensiones inv
álidas.");
        this.filas = filas;
        this.columnas = columnas;
        this.datos = new double[filas][columnas];
    }

    public Matriz2D(double[][] origen) {
        if (origen == null || origen.length == 0 || origen[0] == null) {
            throw new IllegalArgumentException("Matriz origen inválida.");
        }
        this.filas = origen.length;
        this.columnas = origen[0].length;
        this.datos = new double[filas][columnas];
        // Copia defensiva profunda
        for (int i = 0; i < filas; i++) {
            this.datos[i] = Arrays.copyOf(origen[i], columnas);
        }
    }

    public Matriz2D sumar(Matriz2D otra) {
        if (this.filas != otra.filas || this.columnas != otra.columnas) {
            throw new IllegalArgumentException("Las dimensiones deben ser idénticas para sum
ar matrices.");
        }
        Matriz2D res = new Matriz2D(this.filas, this.columnas);
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                res.datos[i][j] = this.datos[i][j] + otra.datos[i][j];
            }
        }
        return res;
    }

    public Matriz2D transponer() {
        Matriz2D t = new Matriz2D(this.columnas, this.filas);
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                t.datos[j][i] = this.datos[i][j];
            }
        }
        return t;
    }

    public double traza() {
        if (this.filas != this.columnas) throw new IllegalStateException("La traza solo se d
efine para matrices cuadradas.");
        double suma = 0;
        for (int i = 0; i < filas; i++) suma += this.datos[i][i];
        return suma;
    }
}
```

Solución Ejercicio 12: Factorías Estáticas con `ColorRGB`

```
import java.util.Objects;

public final class ColorRGB {
    public static final ColorRGB ROJO = new ColorRGB(255, 0, 0);
    public static final ColorRGB VERDE = new ColorRGB(0, 255, 0);
    public static final ColorRGB AZUL = new ColorRGB(0, 0, 255);
    public static final ColorRGB BLANCO = new ColorRGB(255, 255, 255);
    public static final ColorRGB NEGRO = new ColorRGB(0, 0, 0);

    private final int r;
    private final int g;
    private final int b;

    // Constructor estrictamente privado
    private ColorRGB(int r, int g, int b) {
        this.r = r;
        this.g = g;
        this.b = b;
    }

    public static ColorRGB fromRGB(int r, int g, int b) {
        validarRango(r);
        validarRango(g);
        validarRango(b);
        return new ColorRGB(r, g, b);
    }

    public static ColorRGB fromHex(String hex) {
        Objects.requireNonNull(hex);
        String limpio = hex.replace("#", "").trim();
        if (limpio.length() != 6) {
            throw new IllegalArgumentException("El código hexadecimal debe tener 6 dígitos:
" + hex);
        }
        int r = Integer.parseInt(limpio.substring(0, 2), 16);
        int g = Integer.parseInt(limpio.substring(2, 4), 16);
        int b = Integer.parseInt(limpio.substring(4, 6), 16);
        return new ColorRGB(r, g, b);
    }

    private static void validarRango(int v) {
        if (v < 0 || v > 255) {
            throw new IllegalArgumentException("Canal de color fuera de rango [0, 255]: " +
v);
        }
    }

    public String toHex() {
        return String.format("#%02X%02X%02X", this.r, this.g, this.b);
    }

    public int getR() { return this.r; }
    public int getG() { return this.g; }
    public int getB() { return this.b; }

    @Override
    public String toString() {
        return String.format("ColorRGB(%d, %d, %d -> %s)", this.r, this.g, this.b, toHex());
    }
}
```

Solución Ejercicio 13: Array de Objetos `Circulo` y Búsqueda del Mayor Área (`mayorArea`)

```
public class Circulo {
    private double radio;

    public Circulo(double radio) {
        if (radio <= 0) {
            throw new IllegalArgumentException("El radio debe ser mayor que cero.");
        }
        this.radio = radio;
    }

    public double getRadio() {
        return this.radio;
    }

    public double calcularArea() {
        return Math.PI * this.radio * this.radio;
    }

    @Override
    public String toString() {
        return String.format("Circulo(radio=%.2f, área=%.2f)", this.radio, calcularArea());
    }

    public static Circulo mayorArea(Circulo[] circulos) {
        if (circulos == null || circulos.length == 0) {
            throw new IllegalArgumentException("El array de círculos no puede ser nulo ni va
cío.");
        }
        Circulo mayor = circulos[0];
        for (int i = 1; i < circulos.length; i++) {
            if (circulos[i].calcularArea() > mayor.calcularArea()) {
                mayor = circulos[i];
            }
        }
        return mayor;
    }

    public static void main(String[] args) {
        Circulo[] lista = {
            new Circulo(2.5),
            new Circulo(7.1),
            new Circulo(1.0),
            new Circulo(9.4),
            new Circulo(4.2)
        };

        Circulo ganador = mayorArea(lista);
        System.out.println("Círculo de mayor área: " + ganador);
    }
}
```

Solución Ejercicio 14: El Contrato `equals()` / `hashCode()`, la Metáfora de los Cajones y `Punto2D`

1. Razonamiento del contrato `equals()` / `hashCode()` y los cajones (*buckets*): Las estructuras basadas en tablas hash (`HashSet`, `HashMap`) organizan sus elementos en **cajones (*buckets*)** identificados por el número entero que devuelve `hashCode()`.

- Al insertar o buscar un objeto, la colección calcula primero su `hashCode()` para ir directamente a su cajón, y **solo compara con `equals()` los objetos que están dentro de ese mismo cajón**.
- Si sobrescribes `equals()` (por ejemplo, dos puntos con iguales coordenadas `x, y`), pero **olvidas sobrescribir `hashCode()`**, cada objeto heredará el `hashCode()` de `Object` (basado en su dirección de memoria física distinta). En consecuencia, dos objetos iguales según `equals()` tendrán distinto `hashCode()`, caerán en cajones diferentes y `HashSet` nunca detectará que son el mismo elemento, admitiendo duplicados o devolviendo `false` en `.contains()`.

2. Implementación completa de `Punto2D` con `suma()`, `toString()`, `equals()` y `hashCode()`:

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

public class Punto2D {
    private double x;
    private double y;

    public Punto2D() {
        this(0.0, 0.0);
    }

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return this.x; }
    public void setX(double x) { this.x = x; }
    public double getY() { return this.y; }
    public void setY(double y) { this.y = y; }

    public Punto2D suma(Punto2D otro) {
        return new Punto2D(this.x + otro.x, this.y + otro.y);
    }

    @Override
    public String toString() {
        return "(" + this.x + ", " + this.y + ")";
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Punto2D otro)) return false;
        return Double.compare(this.x, otro.x) == 0
                && Double.compare(this.y, otro.y) == 0;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.x, this.y);
    }

    public static void main(String[] args) {
        Punto2D p1 = new Punto2D(3.0, 2.0);
        Punto2D p2 = new Punto2D(1.0, -1.0);
        Punto2D p3 = p1.suma(p2);

        System.out.println(p1 + " + " + p2 + " = " + p3); // (3.0, 2.0) + (1.0, -1.0) = (4.
0, 1.0)

        Set<Punto2D> conjunto = new HashSet<>();
        conjunto.add(new Punto2D(3.0, 2.0));
        System.out.println("¿Conjunto contiene (3.0, 2.0)?: " + conjunto.contains(p1)); // t
rue gracias a hashCode()
    }
}
```

Solución Ejercicio 15: De Clase Mutable `Alumno` a `AlumnoInmutable`

```
import java.util.Arrays;
import java.util.Objects;
import java.util.Random;

// 1. Versión Mutable encapsulada
class Alumno {
    private int nia;
    private String nombre;
    private double[] notas;

    public Alumno(int nia, String nombre, double[] notas) {
        this.nia = nia;
        this.nombre = Objects.requireNonNull(nombre);
        this.notas = (notas != null) ? notas.clone() : new double[3];
    }

    public Alumno(String nombre) {
        this(new Random().nextInt(900000) + 100000, nombre, new double[]{0.0, 0.0, 0.0});
    }

    public int getNia() { return this.nia; }
    public void setNia(int nia) { this.nia = nia; }
    public String getNombre() { return this.nombre; }
    public void setNombre(String nombre) { this.nombre = Objects.requireNonNull(nombre); }
    public double[] getNotas() { return this.notas; }
    public void setNotas(double[] notas) { this.notas = notas; }

    public double notaMedia() {
        if (this.notas == null || this.notas.length == 0) return 0.0;
        double suma = 0;
        for (double n : this.notas) suma += n;
        return suma / this.notas.length;
    }

    @Override
    public String toString() {
        return String.format("{nia: %d, nombre: '%s', notas: %s, media: %.2f}",
                this.nia, this.nombre, Arrays.toString(this.notas), notaMedia());
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Alumno otro)) return false;
        return this.nia == otro.nia;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.nia);
    }
}

// 2. Versión 100% Inmutable (final class + private final + sin setters + copias defensivas
.clone())
public final class AlumnoInmutable {
    private final int nia;
    private final String nombre;
    private final double[] notas;
```

```
    public AlumnoInmutable(int nia, String nombre, double[] notas) {
        this.nia = nia;
        this.nombre = Objects.requireNonNull(nombre);
        // Copia defensiva en el constructor
        this.notas = (notas != null) ? notas.clone() : new double[0];
    }

    public int getNia() { return this.nia; }
    public String getNombre() { return this.nombre; }

    public double[] getNotas() {
        // Copia defensiva en el getter
        return this.notas.clone();
    }

    public double notaMedia() {
        if (this.notas.length == 0) return 0.0;
        double suma = 0;
        for (double n : this.notas) suma += n;
        return suma / this.notas.length;
    }

    @Override
    public String toString() {
        return String.format("AlumnoInmutable{nia=%d, nombre='%s', notas=%s}",
                this.nia, this.nombre, Arrays.toString(this.notas));
    }
}
```

Solución Ejercicio 16: Sistema de Gestión de Inventario: Clases de Dominio, Invariantes y Contenedor de Objetos

```
import java.util.Arrays;
import java.util.Objects;

public class GestionInventarioAlmacen {

    // 1. CLASE DE ENTIDAD: ARTÍCULO
    public static class Articulo {
        private final String codigo; // Inmutable una vez asignado
        private String descripcion;
        private double precio;
        private int stock;

        public Articulo(String codigo, String descripcion, double precio, int stock) {
            if (codigo == null || !codigo.matches("^[A-Z]{3}-\\d{3}$")) {
                throw new IllegalArgumentException("Código inválido (formato esperado 'ABC-1
23'): " + codigo);
            }
            if (descripcion == null || descripcion.isBlank()) {
                throw new IllegalArgumentException("La descripción no puede estar vacía.");
            }
            if (precio <= 0) {
                throw new IllegalArgumentException("El precio debe ser estrictamente positiv
o: " + precio);
            }
            if (stock < 0) {
                throw new IllegalArgumentException("El stock no puede ser negativo: " + stoc
k);
            }
            this.codigo = codigo;
            this.descripcion = descripcion.trim();
            this.precio = precio;
            this.stock = stock;
        }

        public String getCodigo() { return codigo; }
        public String getDescripcion() { return descripcion; }
        public double getPrecio() { return precio; }
        public int getStock() { return stock; }

        public void setDescripcion(String descripcion) {
            if (descripcion == null || descripcion.isBlank()) {
                throw new IllegalArgumentException("Descripción vacía.");
            }
            this.descripcion = descripcion.trim();
        }

        public void setPrecio(double precio) {
            if (precio <= 0) {
                throw new IllegalArgumentException("Precio debe ser > 0.");
            }
            this.precio = precio;
        }

        public void reponer(int cantidad) {
            if (cantidad <= 0) {
                throw new IllegalArgumentException("La cantidad a reponer debe ser positiva:
" + cantidad);
            }
            this.stock += cantidad;
        }
```

```
        public boolean vender(int cantidad) {
            if (cantidad <= 0 || cantidad > this.stock) {
                return false; // Venta rechazada
            }
            this.stock -= cantidad;
            return true;
        }

        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof Articulo otro)) return false;
            return Objects.equals(this.codigo, otro.codigo);
        }

        @Override
        public int hashCode() {
            return Objects.hash(this.codigo);
        }

        @Override
        public String toString() {
            return String.format("Articulo[%s: %s | %.2f € | Stock: %d]",
                    codigo, descripcion, precio, stock);
        }
    }

    // 2. CLASE CONTENEDORA: GESTOR DE ALMACÉN
    public static class GestorAlmacen {
        private final Articulo[] articulos;
        private int totalArticulos;

        public GestorAlmacen(int capacidadMaxima) {
            if (capacidadMaxima <= 0) {
                throw new IllegalArgumentException("La capacidad debe ser positiva.");
            }
            this.articulos = new Articulo[capacidadMaxima];
            this.totalArticulos = 0;
        }

        public boolean registrarArticulo(Articulo a) {
            if (a == null) return false;
            if (totalArticulos >= articulos.length) return false; // Lleno
            if (buscarPorCodigo(a.getCodigo()) != null) return false; // Ya existe

            articulos[totalArticulos++] = a;
            return true;
        }

        public Articulo buscarPorCodigo(String codigo) {
            if (codigo == null) return null;
            for (int i = 0; i < totalArticulos; i++) {
                if (articulos[i].getCodigo().equalsIgnoreCase(codigo)) {
                    return articulos[i];
                }
            }
            return null;
        }

        public boolean eliminarArticulo(String codigo) {
```

```
            for (int i = 0; i < totalArticulos; i++) {
                if (articulos[i].getCodigo().equalsIgnoreCase(codigo)) {
                    // Compactar hacia la izquierda
                    for (int j = i; j < totalArticulos - 1; j++) {
                        articulos[j] = articulos[j + 1];
                    }
                    articulos[totalArticulos - 1] = null; // Limpiar para Garbage Collector
                    totalArticulos--;
                    return true;
                }
            }
            return false;
        }

        public double calcularValorTotalInventario() {
            double total = 0.0;
            for (int i = 0; i < totalArticulos; i++) {
                total += articulos[i].getPrecio() * articulos[i].getStock();
            }
            return total;
        }

        // Copia defensiva del array disponible
        public Articulo[] getArticulosDisponibles() {
            return Arrays.copyOf(this.articulos, this.totalArticulos);
        }

        public int getTotalArticulos() { return totalArticulos; }
        public int getCapacidadMaxima() { return articulos.length; }
    }

    public static void main(String[] args) {
        GestorAlmacen almacen = new GestorAlmacen(5);

        Articulo a1 = new Articulo("TEC-101", "Teclado Mecánico RGB", 59.99, 15);
        Articulo a2 = new Articulo("RAT-202", "Ratón Óptico Ergonómico", 29.50, 20);
        Articulo a3 = new Articulo("MON-303", "Monitor 27 Pulgadas IPS", 189.00, 8);

        almacen.registrarArticulo(a1);
        almacen.registrarArticulo(a2);
        almacen.registrarArticulo(a3);

        System.out.println("Artículos registrados: " + almacen.getTotalArticulos());
        System.out.printf("Valor total del inventario: %.2f €%n", almacen.calcularValorTotal
Inventario());

        // Operaciones de venta
        System.out.println("\nVenta de 5 teclados: " + (a1.vender(5) ? "Éxito" : "Rechazad
a"));
        System.out.println("Venta de 50 monitores (insuficiente): " + (a3.vender(50) ? "Éxit
o" : "Rechazada"));
        System.out.println(a1);

        // Eliminación y compactación
        System.out.println("\nEliminando ratón (RAT-202): " + almacen.eliminarArticulo("RAT-
202"));
        System.out.println("Total artículos tras borrado: " + almacen.getTotalArticulos());

        // Listar copia defensiva
        System.out.println("\nListado actual del almacén:");
        for (Articulo a : almacen.getArticulosDisponibles()) {
```

```
            System.out.println(" - " + a);
        }
    }
}
```